

SCHOOL ORGANIZATION EVENT MANAGER

Java Console Application | Midterm Laboratory Examination

Purpose

Help school event managers maintain event information and registration counts while attendees discover and join eligible events.

Prepared for the Midterm Laboratory Examination

Student name: _____ Section: _____

8 October 2026 | Video demonstration excluded as requested

1. Problem definition

Problem statement

A school organization may announce several activities, each with its own schedule, venue, leader, age rule, and limited number of places. Managers can lose track of changing details and attendance when they use separate notes or message threads. Attendees need a clear way to see current events and register without accidentally joining twice or exceeding a capacity limit. This application keeps event information, accounts, and registrations together in one console session.

Target users

- **Event managers** create accounts, create and maintain their own events, view registration counts, and see registrant labels.
- **Attendees** create accounts, browse event details, register when eligible, and cancel their own registrations.

Objectives

- Keep titles, descriptions, dates, locations, leaders, minimum ages, capacities, and status in one place.
- Apply age, capacity, duplicate, canceled-event, and start-time rules before adding a registration.
- Calculate attendance counts from active registration records instead of maintaining a second count field.

Scope and limitations

The program is a single-computer, in-memory classroom prototype. It has no database, network access, notifications, or persistent login. Passwords are held as plain strings in memory; users should use sample passwords only. Event times follow the computer's local time zone. Canceled events remain visible with their existing registrations for review. Event starts and minimum ages become locked once registration begins; capacity cannot be reduced below the current count.

Final proposed solution

The opening menu asks for Attendee or Event Manager. Each role has account creation and login, followed by its own menu. One attendee account uses `Lastname, Firstname`. Each registration displays `Lastname, Firstname-Event Title`, so the same account can join multiple events without acquiring conflicting account names. This resolves the open naming question while keeping a manager-friendly event roster.

2. OOP analysis and design

The design separates identity, event data, registration data, and the rules that coordinate them. Important state is private; methods check changes before updating it.

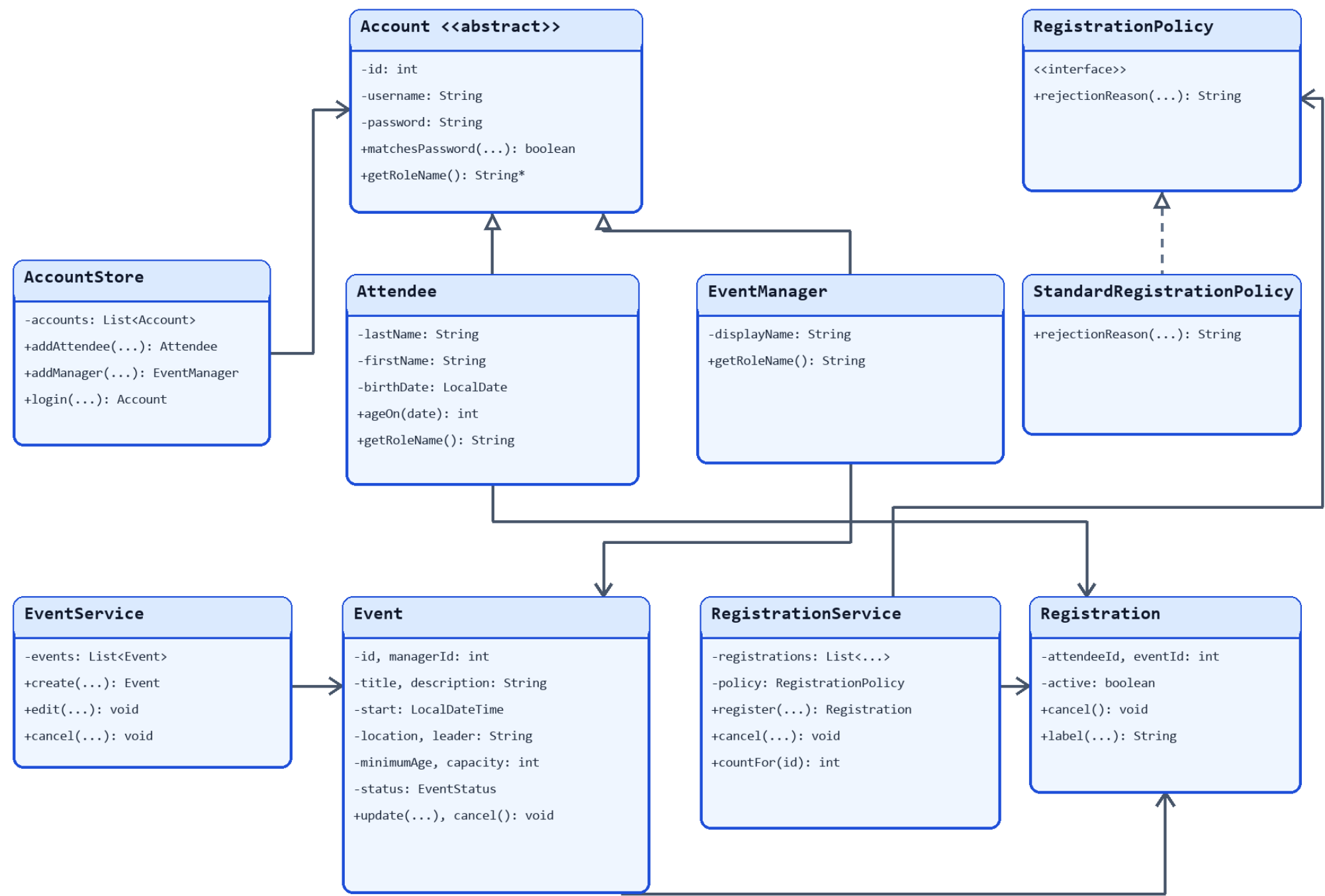
Classes, attributes, methods, and responsibilities

Class	Key private state	Main methods	Responsibility
Account (abstract)	id, username, password	matchesPassword, getRoleName	Common account identity and role contract
Attendee	firstName, lastName, birthDate	ageOn, getRoleName	Attendee identity and age calculation
EventManager	displayName	getRoleName	Manager identity
AccountStore	accounts, nextId	addAttendee, addManager, login	Create unique accounts and authenticate
Event	title, description, start, location, leader, minimumAge, capacity, status	update, cancel, getters	Validate and hold one event's state
EventService	events, clock, nextId	create, edit, cancel, byManager	Check event ownership and timing
Registration	attendeeld, eventId, active	cancel, label	Represent one attendee/event link
RegistrationService	registrations, policy, clock	register, cancel, countFor	Apply policy and derive counts
RegistrationPolicy (interface)	none	rejectionReason	Contract for admission rules
StandardRegistrationPolicy	none	rejectionReason	Age, duplicate, capacity, timing checks

Relationships: Attendee and EventManager *are* Accounts. AccountStore has many Accounts; EventService has many Events; RegistrationService has many Registrations and one RegistrationPolicy. A manager owns multiple events. An attendee can make many registrations; each registration refers to one event and one attendee.

3. UML class diagram

Visibility: - private; + public. The full editable Mermaid source is also included in the package.



Hollow triangle = inheritance Dashed line = interface implementation Open arrow = association
Cardinality: stores/services contain 0..* records; each registration refers to 1 attendee and 1 event.

Class diagram of the principal types and their relationships. EventStatus is a two-value enum: SCHEDULED or CANCELED.

4. How the required Java concepts are used

Input, conditions, and loops

Main uses Scanner for typed console input. Main and role menus repeat with while loops and switch choices. Integer and date inputs are retried after invalid entries. Conditions in the registration policy choose the correct rejection message.

Encapsulation and controlled access

Account, Event, and Registration fields are private. Getters expose information needed for display and rules. Changes go through methods such as Event.update() and Registration.cancel(), keeping validation in one place. The Java keyword this distinguishes fields from constructor parameters.

Inheritance, overriding, super, and polymorphism

Account is an abstract superclass for Attendee and EventManager. Their constructors call super(...) to initialize shared account fields. Each overrides getRoleName(). AccountStore stores Account references and calls the same method on either subclass; the executed version depends on the object type.

```
public abstract class Account {
    private final String username;
    public abstract String getRoleName();
}

public final class EventManager extends Account {
    public EventManager(int id, String username, String name, String password) {
        super(id, username, password);
        this.displayName = name.trim();
    }
    @Override public String getRoleName() { return "Event Manager"; }
}
```

Abstraction and interface

RegistrationPolicy is an interface for deciding whether registration is allowed. StandardRegistrationPolicy implements the actual school-event rules. RegistrationService depends on the interface, so another policy could be supplied without rewriting registration storage.

```
String reason = policy.rejectionReason(attendee, event, registrations, now);
if (reason != null) throw new IllegalArgumentException(reason);
registrations.add(new Registration(attendee.getId(), event.getId(), now));
```

5. Development process and AI evaluation

Brainstorming summary

The starting idea was a school organization's event list with manager editing. The design was refined to include two account roles, attendee registration and cancellation, age rules, capacity, duplicate prevention, and manager-visible counts. The multi-event account-name ambiguity was resolved before the UML was finalized.

Three AI suggestions evaluated

Suggestion considered	Evaluation	Decision
Use Lastname,Firstname-Event as the account name.	One attendee joining two events would need two names or a misleading first-event suffix.	Modified: use one Lastname,Firstname account and an event-specific registration label.
Keep a numeric attendance count on Event.	A second stored count could drift from cancellations and repeat registrations.	Rejected: count active Registration objects when displaying the event.
Use inheritance for the two account roles and an interface for registration rules.	Attendee and EventManager really are accounts; registration rules are behavior that can vary independently.	Accepted: abstract Account with two subclasses; RegistrationPolicy interface.

Problems encountered and improvements

- **Multi-event identity:** moved the event suffix to the registration label so a person can use one account.
- **Age boundary:** age is calculated on the event date, not the registration date. Tests 9 and 10 cover the birthday boundary.
- **Editing with existing registrations:** start time and minimum age are locked after the first registration, and capacity cannot fall below the active count.

How ChatGPT helped

AI assistance was used to organize the OOP design, draft code and documentation, and propose boundary tests. The 24 repeatable tests were then compiled and run against the actual Java implementation. The student's final submission should be reviewed line by line and explained independently.

What I personally learned - draft for student review

A useful lesson from this project is the difference between an is-a relationship (an Attendee is an Account) and a has-a relationship (an EventService has Events). I also saw why a derived registration count is safer than updating two separate pieces of state. **Rewrite this paragraph in your own words before submitting it as a personal reflection.**

6. Test cases (1 of 2)

Automated Java SelfTest uses a fixed clock so age and timing boundaries remain repeatable. All cases below were executed; see the captured test output.

#	Feature	Input	Expected result	Actual result	Result
1	Attendee account/login	Reyes; Ana; 2005-05-01; secret1	Username and login succeed	Username and login succeeded	Pass
2	Role polymorphism	Attendee and EventManager objects	Distinct role names via Account references	Attendee and Event Manager returned	Pass
3	Duplicate username	reyes,ana after Reyes,Ana	Reject case-insensitive duplicate	Duplicate rejected	Pass
4	Incorrect password	lead; wrong	Login fails	Login returned no account	Pass
5	Minimum capacity	Age 0; capacity 1	Event created with boundary values	Event created with boundary values	Pass
6	Empty title	Whitespace title	Reject event	Event rejected	Pass
7	Invalid capacity	Capacity 0	Reject event	Event rejected	Pass
8	Past event	2025-01-01 09:00 with fixed 2026 clock	Reject past start	Past start rejected	Pass
9	Minimum age boundary	Age exactly 18 on event day; minimum 18	Accept registration	Registration accepted	Pass
10	Below minimum age	Age 17 on event day; minimum 18	Reject registration	Registration rejected	Pass
11	Duplicate registration	Same attendee registers twice	Second attempt rejected and count stays 1	Second attempt rejected and count stayed 1	Pass
12	Capacity reached	Capacity 1; second attendee	Reject second attendee	Second attendee rejected	Pass

Result: 24 passed, 0 failed in the recorded test run.

6. Test cases (2 of 2)

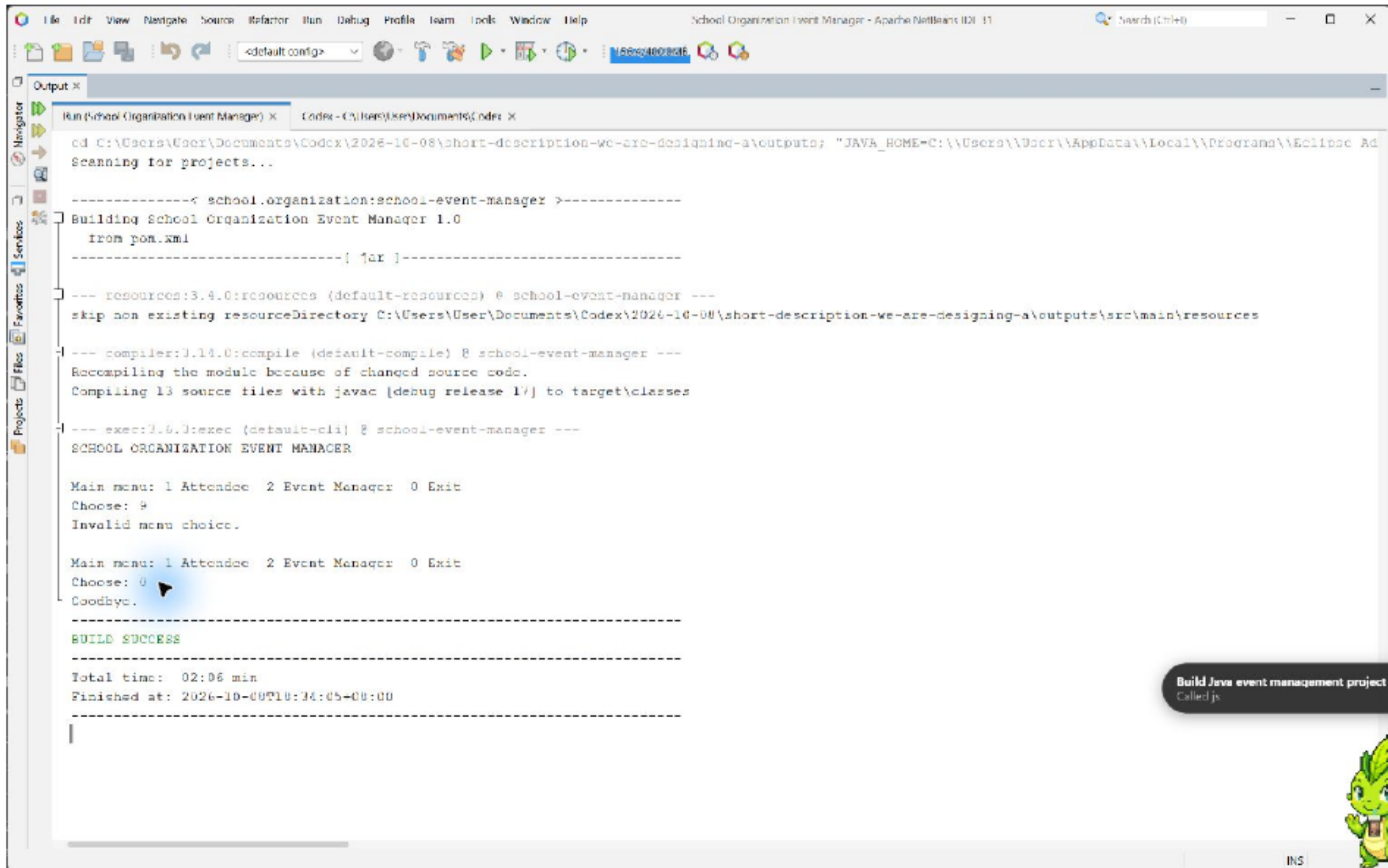
Automated Java SelfTest uses a fixed clock so age and timing boundaries remain repeatable. All cases below were executed; see the captured test output.

#	Feature	Input	Expected result	Actual result	Result
13	Cancel and repeat	Register; cancel; register again	Count changes 1 to 0 to 1	Count changed 1 to 0 to 1	Pass
14	Registration label	Reyes Ana joins School Fair	Reyes,Ana-School Fair	Correct label returned	Pass
15	Canceled event	Attempt registration after manager cancels	Reject registration	Registration rejected	Pass
16	Started event	Attempt registration after start	Reject registration	Registration rejected	Pass
17	Event ownership	Other manager edits event	Reject unauthorized edit	Edit rejected	Pass
18	Capacity edit boundary	Two registrations then capacity 1	Reject reduction	Reduction rejected	Pass
19	Eligibility change after registration	Change event start and minimum age	Reject change	Change rejected	Pass
20	Multiple events	One attendee joins two events	Two registrations under one account	Two registrations under one account	Pass
21	Incorrect menu selection	9 at main menu	Show invalid choice and continue	Invalid choice shown and main menu repeated	Pass
22	Empty required input	Blank manager username then lead	Show error and retry username	Error shown and account created after retry	Pass
23	Upper boundary	Minimum age 120; capacity 1000	Accept valid upper and reasonable capacity values	Event created with both values	Pass
24	Above maximum age rule	Minimum age 121	Reject event	Event rejected	Pass

Result: 24 passed, 0 failed in the recorded test run.

7. NetBeans interactive execution

Apache NetBeans IDE 31 was installed and used to open the Maven project on 8 October 2026. This genuine IDE screenshot shows Main running, rejecting menu choice 9, accepting Exit, and finishing with BUILD SUCCESS.



The screenshot displays the Apache NetBeans IDE 31 interface. The main window shows the 'Output' tab for the 'Run (School Organization Event Manager)' goal. The output text is as follows:

```
cd C:\Users\User\Documents\Codex\2026-10-08\short-description-we-are-designing-a\outputs; "JAVA_HOME=C:\Users\User\AppData\Local\Programs\Eclipse Ad
Scanning for projects...

-----< school.organization:school-event-manager >-----
Building School Organization Event Manager 1.0
from pom.xml
-----[ jar ]-----

--- resources:3.4.0:resources (default-resources) @ school-event-manager ---
skip non existing resourceDirectory C:\Users\User\Documents\Codex\2026-10-08\short-description-we-are-designing-a\outputs\src\main\resources

--- compiler:3.14.0:compile (default-compile) @ school-event-manager ---
Recompiling the module because of changed source code.
Compiling 13 source files with javac [debug release 17] to target\classes

--- exec:3.6.3:exec (default-cli) @ school-event-manager ---
SCHOOL ORGANIZATION EVENT MANAGER

Main menu: 1 Attendee  2 Event Manager  0 Exit
Choose: 9
Invalid menu choice.

Main menu: 1 Attendee  2 Event Manager  0 Exit
Choose: 0
Goodbye.

BUILD SUCCESS

Total time: 02:06 min
Finished at: 2026-10-08T10:34:05+00:00
```

A mouse cursor is visible over the 'Choose: 0' line. In the bottom right corner, there is a dark grey tooltip that reads 'Build Java event management project' and 'Called js'. A small green cartoon character is visible in the bottom right corner of the IDE window.

8. Source and test results in NetBeans

The restored SelfTest.java was opened in the IDE and executed with Run File (Shift+F6). This fresh screenshot shows the source tree, test code, 24 passing cases, and successful build.

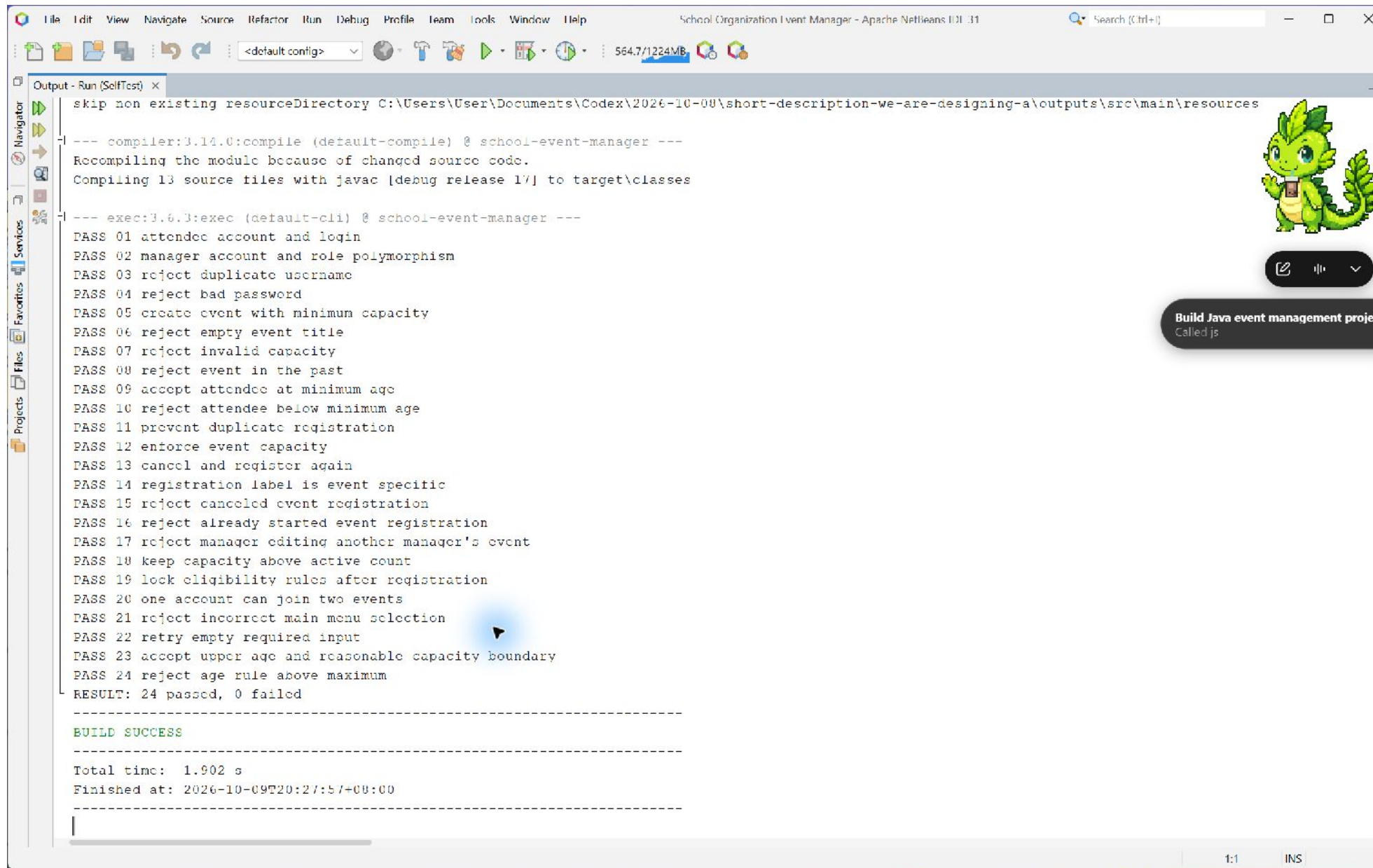
The screenshot displays the Apache NetBeans IDE interface. The top toolbar shows various icons for file operations and running the application. The left sidebar contains the 'Projects' view, showing the 'School Organization Event Manager' project with its source packages and files. The 'SelfTest.java' file is selected, and its source code is displayed in the main editor. The code includes imports for various Java classes and a main method that runs a series of tests. The bottom right pane shows the 'Output - Run (SelfTest)' window, which displays the results of the test run, including 24 passing cases and a successful build.

```
1  /*
2  * Decompiled with CFR 0.152.
3  */
4  import java.io.OutputStream;
5  import java.nio.charset.StandardCharsets;
6  import java.nio.file.Path;
7  import java.time.Clock;
8  import java.time.Instant;
9  import java.time.LocalDate;
10 import java.time.LocalDateTime;
11 import java.time.ZoneOffset;
12
13 public final class SelfTest {
14     private static final Clock CLOCK = Clock.fixed(Instant.parse("2026-01-01T10:00:00Z"), ZoneOffset.UTC);
15     private static final LocalDateTime FUTURE = LocalDateTime.of(2027, 1, 1, 9, 0);
16     private static int passed;
17     private static int failed;
18
19     public static void main(String[] stringArray) {
20         SelfTest.test("01 attendee account and login", () -> {
21             AccountStore accountStore = new AccountStore();
22             Attendee attendee = accountStore.addAttendee("Reyes", "Ana", LocalDate.of(2005, 5, 1), "secret1");
23             SelfTest.check(attendee.getUsername().equals("Reyes,Ana"));
24             SelfTest.check(accountStore.login("reyes,ana", "secret1", Attendee.class) == attendee);
25         });
26         SelfTest.test("02 manager account and role polymorphism", () -> {
27             Attendee attendee = new Attendee(1, "Reyes", "Ana", LocalDate.of(2005, 5, 1), "secret1");
28             EventManager eventManager = new EventManager(2, "manager", "Club Lead", "secret2");
```

Output - Run (SelfTest) x

```
PASS 23 accept upper age and reasonable capacity boundary
PASS 24 reject age rule above maximum
RESULT: 24 passed, 0 failed
-----
BUILD SUCCESS
-----
Total time: 1.902 s
Finished at: 2026-10-09T20:27:57+08:00
-----
```

9. Current NetBeans test execution: 24 cases

The screenshot shows the NetBeans IDE interface. The main window displays the 'Output - Run (SelfTest)' console. The output text is as follows:

```
skip non existing resourceDirectory C:\Users\User\Documents\Codex\2026-10-08\short-description-we-are-designing-a\outputs\src\main\resources  
  
--- compiler:3.14.0:compile (default-compile) @ school-event-manager ---  
Recompiling the module because of changed source code.  
Compiling 13 source files with javac [debug release 17] to target\classes  
  
--- exec:3.6.3:exec (default-cli) @ school-event-manager ---  
PASS 01 attendee account and login  
PASS 02 manager account and role polymorphism  
PASS 03 reject duplicate username  
PASS 04 reject bad password  
PASS 05 create event with minimum capacity  
PASS 06 reject empty event title  
PASS 07 reject invalid capacity  
PASS 08 reject event in the past  
PASS 09 accept attendee at minimum age  
PASS 10 reject attendee below minimum age  
PASS 11 prevent duplicate registration  
PASS 12 enforce event capacity  
PASS 13 cancel and register again  
PASS 14 registration label is event specific  
PASS 15 reject canceled event registration  
PASS 16 reject already started event registration  
PASS 17 reject manager editing another manager's event  
PASS 18 keep capacity above active count  
PASS 19 lock eligibility rules after registration  
PASS 20 one account can join two events  
PASS 21 reject incorrect main menu selection  
PASS 22 retry empty required input  
PASS 23 accept upper age and reasonable capacity boundary  
PASS 24 reject age rule above maximum  
RESULT: 24 passed, 0 failed  
  
BUILD SUCCESS  
  
Total time: 1.902 s  
Finished at: 2026-10-09T20:27:57+00:00
```

On the right side of the IDE, there is a 'Build Java event management project' notification bubble with a green dragon icon and the text 'Called js'. The bottom status bar shows '1:1' and 'INS'.

This genuine NetBeans capture shows 24 passed, 0 failed, and BUILD SUCCESS. The Java 17 command-line result is also saved in the evidence folder.

Conclusion

The application meets the requested console and OOP scope: managers maintain their own events, attendees use accounts to join and cancel, and registration rules protect age limits, capacity, duplicate entries, canceled events, and started events. Counts are calculated from active registrations. The source, UML, test table, README, and execution evidence are included. A timed video presentation script is provided separately; the student must record the demonstration.

Submission reminder: record your own demonstration, export or share the complete ChatGPT conversation separately, add your name and section, and replace the draft personal reflection with your own account of what you learned.